

SortedSet (Interface) $\Rightarrow$ (extends Set interface)

SortedSet is the alternate of Set interface that provides a total ordering on its elements.

The elements of the SortedSet are arranged in the increasing (ascending) order.

The SortedSet provides the additional methods that inhibit the natural ordering of the elements. (restrict & prevent)

The SortedSet can be instantiated as :

SortedSet <data-type> set = new TreeSet();

TreeSet (class) $\Rightarrow$ (implements Set)

Java TreeSet class implements the Set interface that uses a tree for storage. Like HashSet, TreeSet also contains unique elements. However, the access and retrieval time of TreeSet is quite fast.

The elements in TreeSet stored in ascending order.

Example :->

```
import java.util.*;

public class TestJavaCollection9 {

    public static void main (String args[]) {

        //Creating and adding elements.

        TreeSet<String> set = new TreeSet<String>
            ();

        set.add ("Ravi");
        set.add ("Vijay");
        set.add ("Ravi");
        set.add ("Ajay");

        // Traversing elements.

        Iterator<String> itr = set.iterator();

        while (itr.hasNext()) {

            System.out.println(itr.next());

        }

    }

}
```

Output :->

Ajay

Ravi

Vijay.

** Map interface $\Rightarrow$

The Map interface in Java is part of the Java Collections Framework. It represents a mapping between a set of keys and their corresponding values.

A Map cannot contain duplicate keys; each key can map to at most one value. The Map interface is used to store key-value pairs, where each key is unique, and it provides an efficient way to remove, update, and manipulate data based on keys.

Syntax $\Rightarrow$

```
Map<T,T> hm = new HashMap<>();  
Map<T,T> tm = new TreeMap<>();
```

HashMap (class) $\Rightarrow$ (implements Map)

HashMap in Java is a key-value data structure offering efficient data access via keys using hashing. Hashing converts large strings or other objects into smaller, consistent values for quick indexing and searching.

HashMap implements the Map interface and is used for managing large datasets efficiently. Additionally, HashSet uses HashMap internally to store elements uniquely, demonstrating the utility of hashing in Java's collections framework for fast data removal and management.

Example :->

```
import java.util.HashMap;
import java.util.Map;

public class HashMapExample {
    public static void main(String args[]) {
        // creating a HashMap.
        Map<String, Integer> map = new HashMap<>();

        // Adding key-value pairs to the HashMap
        map.put("Alice", 10);
        map.put("Bob", 20);
        map.put("Charlie", 30);

        // Removing a value.
        System.out.println("value for 'Alice' " + map.get("Alice"));

        // iterating over key-value pairs.
        for (Map.Entry<String, Integer> entry :
             map.entrySet()) {
            String key = entry.getKey();
            Integer value = entry.getValue();
        }
    }
}
```



```
} System.out.println(key + ":" + value);
```

```
// Removing a key-value pair.
```

```
map.remove("Charlie");
```

```
// checking the presence of a key.
```

```
if (map.containsKey("Bob")) {
```

```
System.out.println("Map contains key 'Bob'.");
```

```
}
```

```
}
```

```
}
```

Output :-

Value for 'Alice' : 10 Bob : 20

Alice : 20

Charlie : 30

Map contains key 'Bob'.

DATE PAGE NO.

* Major Difference between in Collection framework $\Rightarrow$

① List vs Set $\Rightarrow$

i) List allows duplicates; Set does not.

ii) List maintains insertion order; Set does not (except LinkedHashSet).

② Set vs Map $\Rightarrow$

i) Set contains only elements, while Map store key-value pairs.

ii) Set ensures uniqueness of elements; Map ensures unique keys.

③ Queue vs Deque $\Rightarrow$

i) Queue is strictly FIFO.

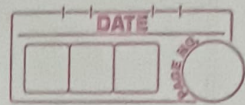
ii) Deque allows operations at both ends (FIFO and LIFO)

④ ArrayList vs LinkedList $\Rightarrow$

i) ArrayList is fast random access ($O(1)$), slower insertions/deletions ($O(n)$).

ii) LinkedList is fast insertions/deletions ($O(1)$), slower random access ($O(n)$).

* Array vs ArrayList in Java :->



Array
(int[], String[],
etc)

ArrayList
(ArrayList<E>)

i> fixed size

i> Dynamic size (resizable)

ii> Can store primitives
(int, char, etc) & objects

ii> Store only objects
(Integer, String, etc).

iii> Faster (direct memory
allocation)

iii> slightly slower
(dynamic resizing).

iv> Only basic operations
(length, index access).

iv> Rich API (add(), remove
contains(), etc).

v> Use loops (for, while)

v> Enhanced loops, Iterator &
Streams.

vi> Can store only one
type, defined at
creation.

vi> Generic type (ArrayList<T>)
safer but required
wrapper classes for
primitives.

Example :->

Example :->

```
int[] number = new int[3];  
// Array creation &  
// declaration
```

```
number[0] = 10;  
number[1] = 20;  
number[2] = 30;  
// Adding  
// element
```

```
ArrayList<Integer> list =  
new ArrayList<>();  
// ArrayList creation & declaration  
list.add(10);  
list.add(20);  
list.add(30);
```


* Comparable vs Comparator in Java :->

Both Comparable and Comparator are used to sort objects in Java, but they have different implementations and use cases.

① Comparable Interface (`java.lang.Comparable`) (Natural Ordering)

i) Used for natural ordering of objects.

ii) A class implements `Comparable<T>` and overrides the `compareTo(T o)` method.

iii) Modified the class itself to define its sorting behaviour.

Example

```
import java.util.*;  
  
class Student implements Comparable<Student> {  
    int id;  
    String name;  
  
    public Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```


// sorting by id (natural order)

```
public int compareTo(Student s) {
```

```
    return this.id - s.id; // Ascending order
                             (sorting logic)
}
```

```
public String toString() {
```

```
    return id + "-" + name;
}
```

```
public class ComparableExample {
```

```
    public static void main(String[] args) {
```

```
        List<Student> students = new ArrayList<>();
```

```
        students.add(new Student(3, "Alice"));
```

```
        students.add(new Student(1, "Bob"));
```

```
        students.add(new Student(2, "Charlie"));
```

```
        Collections.sort(students); // use
                                         Comparable
```

```
        System.out.println(students); // output
```

```
    }
```

```
}
```

Output →

[1 - Bob, 2 - Charlie, 3 - Alice]

② Comparator Interface (java.util.Comparator) (Custom Sorting)

i) Used for custom ordering of objects

ii) Sorting logic is defined outside the class.

iii) Multiple sorting strategies can be implemented without modifying the model class.

iv) Required implementing `compare()` method

Example $\Rightarrow$

```
import java.util.*;
```

```
class Student {
```

```
    int id;
```

```
    String name;
```

```
    public Student(int id, String name) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
    }
```



```

public String toString() {
    return id + "-" + name;
}

// Comparator for sorting by name
class NameComparator implements
    Comparator<Student> {
    public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name);
        // Alphabetical order.
    }
}

```

```

public class ComparatorExample {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();

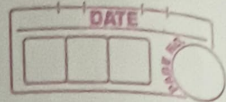
        students.add(new Student(3, "Alice"));
        students.add(new Student(1, "Bob"));
        students.add(new Student(2, "Charlie"));

        Collections.sort(students, new NameComparator());
        // use comparator
        System.out.println(students);
    }
}

```

Output → [3-Alice, 1-Bob, 2-Charlie].

(T represents the type of object)



Comparable
(compareTo())

Comparator
(compare())

i) inside the class | i) Outside the class

ii) java.lang.Comparable | ii) java.util.Comparator

iii) int compareTo(T obj) | iii) int compare(T obj1, T obj2)

iv) Single sorting method (natural order) | iv) Multiple sorting strategies

v) Modified original class

v) It does not modify original class

vi) Sorting by ID in a model class

vi) Sorting by name, age, or other fields extremely.

vii) Natural Sorting (e.g. sorting employees by their ID)

vii) Custom Sorting (e.g. sorting employees by name, salary, etc.)

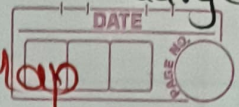
* When to use what?

i) Use Comparable when there is a single natural sorting order (e.g. sorting employees by ID)

ii) Use Comparator when multiple sorting options are needed (e.g. sorting employees by name, age, salary).

*** (IMP) (Add & java & code with durgesh)

* Internal Working of HashMap



```
HashMap<Integer, String> map = new HashMap<>();
map.put(101, "Ankit");
```

When we call put method then put method internally calls putval, then putval internally calls hash method, then hash internally calls hashCode method.

Then the hashCode is calculated of that particular key, Then Index calculated with the help of hashCode.

map.put(101, "Ankit")

putval(key)

hash(key)

hashCode(key)

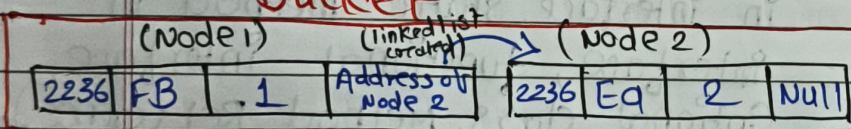
When Index calculated then it is assigned to that particular index

Now lets understand Hash collision

Suppose we put("FB", 1) & put("Ea", 2) in this case both Hashcode is same

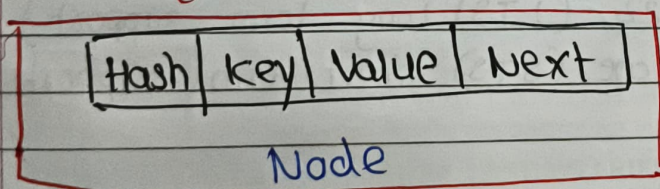
Then in one bucket both the index assign

Bucket



(This is Hash collision)

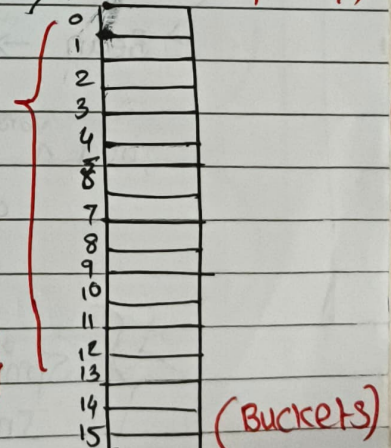
Each Index contains a Node in form of



Node

Bucket

with the help of Hashcode index at key calculated internally 16 (initial capacity)

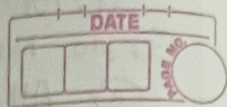


(Buckets)

Threshold = capacity * load factor

Threshold = 16 * 0.75 = 12
(When we cross 12 element then its capacity increases)

Here when the number of elements exceeds 16 x 0.75 = 12, the capacity will double to 32



* Capacity & Load factor $\Rightarrow$

In java, capacity and Load factor are terms primarily used in the context of hash based collections like HashMap and HashSet.

① Capacity $\Rightarrow$ (No. of available slots in hash table)

Capacity refers to the number of buckets (or slots) available in a hash table.

② Load factor $\Rightarrow$ (The fraction of capacity at which resizing occurs)

The load factor determines how full a hash table can get before resizing (reallocating) occurs.

Formula $\Rightarrow$

$$\text{Threshold} = \text{Capacity} \times \text{load factor}$$

where

initial capacity is 16 and load factor is 0.75. (Default)

code $\rightarrow$

```
// Creating a HashMap
HashMap<Integer, String> map = new HashMap
<> (16, 0.75F);
map.put (1, "one");
:
:
System.out.println ("HashMap: " + map);
```